

# Report - Version 1: Character Statistical Model

## Introduction

Version 1 is the first working model in the Building LLM project.

The goal is to build a simple and interpretable language model from scratch, using standard Python only and without neural networks or external frameworks.

This version focuses on the fundamental flow of a character model:

```
Training text
↓
Character pairs
↓
Transition counts
↓
Probabilities
↓
Sampling
↓
Generated text
```

The structure is intentionally minimal: every step remains visible and can be inspected directly.

---

## Goal of Version 1

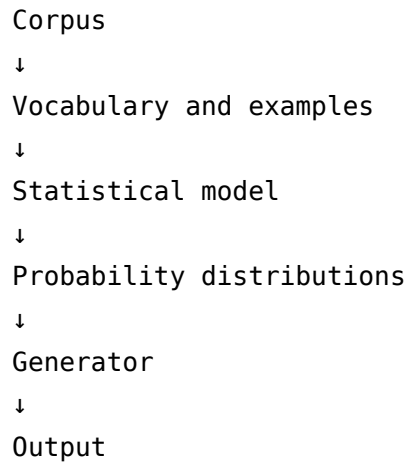
The model should be able to:

- read a small English corpus;
- build the character vocabulary;
- create current-character and next-character examples;
- learn transition counts;
- convert counts into probabilities;
- generate text by sampling one character at a time.

This version is the educational foundation of the project.

# Architecture

The complete Version 1 flow is:



The model uses a single character as context. For every observed character, it stores which characters appeared immediately after it and how often.

Generation is autoregressive: the newly generated character becomes the context for the next step.

---

## Corpus and vocabulary

The corpus is a short English text embedded directly in the notebook. No external files are required.

The vocabulary is the set of all distinct symbols in the corpus. In a character model, letters, spaces, punctuation marks, and newlines are tokens.

```
vocabulary = sorted(set(corpus))
```

```
Vocabulary size: 27
```

```
'\n ,.abcdefghijklmnopqrstuvwxyz'
```

## Training examples

Every adjacent character pair is one example. The first character is the context and the second is the target to predict.

For the fragment model, the examples are:

```
m -> o
o -> d
d -> e
e -> l
```

In the notebook, examples are created by pairing the corpus with a copy shifted by one position.

```
examples = list(zip(corpus, corpus[1:]))
```

A corpus of 440 characters produces 439 examples because the final character has no following character.

---

## Statistical model

Training consists of counting observed transitions. For every current character, the model keeps a counter of possible following characters.

```
for current_character, next_character in zip(text, text[1:]):
    counts[current_character][next_character] += 1
```

The count table is the learned model. There are no neural weights, embeddings, or gradient-based optimization yet.

## Probabilities

To obtain a distribution, each count is divided by the total number of transitions observed from the same current character.

$$P(\text{next} \mid \text{current}) = \frac{\text{count}(\text{current} \rightarrow \text{next})}{\text{total counts}(\text{current})}$$

The probabilities for one context sum to 1. If a character has no transitions, the function returns an empty dictionary.

For the character m, the corpus produces:

o: 4/18    a: 4/18    e: 4/18  
space: 2/18    p: 2/18    s: 2/18

---

## Sampling

The generator does not always select the same character. It samples from the learned distribution: characters with higher probability are more likely to be selected.

```
random_generator.choices(  
    characters,  
    weights=weights,  
    k=1  
)[0]
```

A local seed makes the result reproducible without changing Python's global random state.

## Text generation

Starting from an initial character, the generator repeats four operations:

- read the current character;
- sample the next character;
- append the character to the output;
- use the new character as the next context.

The function checks that the requested length is at least 1 and that the starting character was observed during training.

```
generate_text(model, start_character="i", length=300, seed=42)
```

The generated text may resemble English locally, but it cannot maintain long-term coherence because the context contains only one character.

---

## Included checks

The notebook assertions verify:

- the correct number of examples;
- next-character probabilities sum to 1;
- generation is reproducible with the same seed;
- the generated output has the requested length.

A complete run ends with the message `All checks passed.`

## What Version 1 teaches

The main idea is:

observe -> count -> normalize -> sample -> repeat

Version 1 shows that a language model can be described as a distribution of the next symbol conditioned on the available context.

It also makes the autoregressive loop concrete. Later versions can preserve this loop while using richer contexts and more powerful models.

---

## Limitations

This first version has intentional limitations:

- the context contains one character only;
- the corpus is small and embedded in the notebook;
- unseen transitions have zero probability;
- the model does not understand words, sentences, or meaning;
- text quality depends entirely on corpus counts.

These limitations define the starting point of the project.

---

## Conclusion

Version 1 is the first working model in Building LLM. It transforms a corpus into examples, learns a transition table, builds probability distributions, and generates new text one character at a time.

The result is simple, fully inspectable, and consistent with the goal of building a language model from first principles.